A2 - Controle de Acesso e Liberação de Catracas

Documentação técnica de arquitetura, configuração, integração e operação

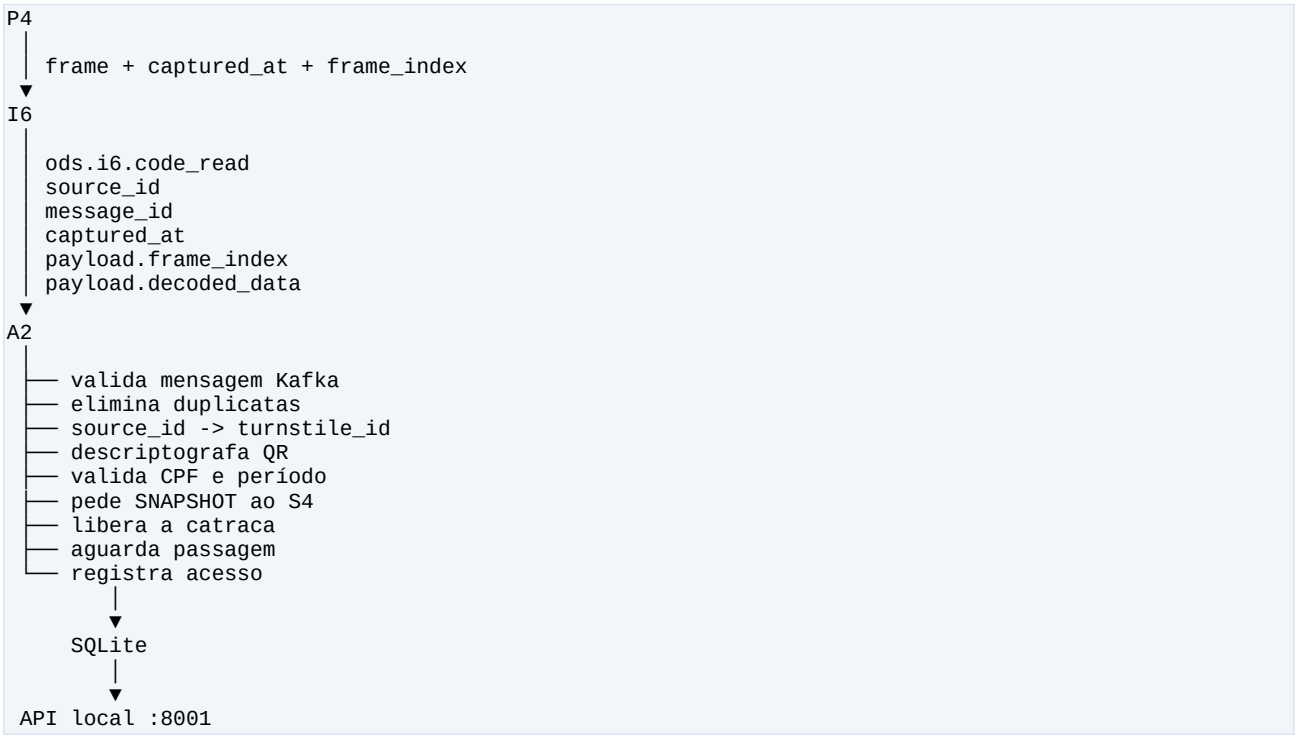
ODS 2026/2 | Componente A2

1. Visão geral

O componente **A2 — Acesso / Identificação** recebe leituras de QR Code produzidas pelo I6, interpreta e valida o código, identifica qual catraca está associada à câmera que realizou a leitura, solicita uma evidência de mídia ao S4, libera a catraca, aguarda a confirmação física da passagem e registra o acesso em um banco SQLite.

O A2 não abre câmeras e não realiza leitura de QR diretamente. Essa responsabilidade é do I6. Também não grava imagens ou vídeos; a evidência de mídia pertence ao S4.

Fluxo geral:



O A2 foi desenvolvido para operar com múltiplas câmeras e múltiplas catracas.

2. Arquivos principais

liberar.py

É o processo principal do A2.

Responsabilidades:

- conectar ao Kafka;
- consumir `ods.i6.code_read`;
- validar o evento recebido;
- fazer deduplicação por `message_id`;
- associar `source_id` a uma catraca;
- manter apenas a leitura pendente mais recente por catraca;
- validar o QR;
- solicitar evidência ao S4;
- comandar a liberação da catraca;
- aguardar confirmação de passagem;
- registrar o acesso no SQLite;
- manter workers de reconexão das catracas.

catraca_driver.py

Contém a abstração dos dispositivos físicos/simulados.

Atualmente existem drivers para:

- `SimulatedTurnstile`;
- `VisualSimulatedTurnstile`;
- `MobileTurnstile`, usado também para o simulador executado em outro computador;
- `ControlIDBlock`, suporte real atual para catracas Control iD.

Para hardware real, o suporte existente atualmente é para **Control iD / iDBlock**.

A arquitetura foi feita para facilitar a inclusão de outros fabricantes. Para isso, normalmente basta implementar um novo driver e adicioná-lo à função `create_turnstile_driver()`.

catraca_monitor.py

Executa o servidor HTTP que recebe callbacks das catracas.

Porta padrão:

```
8000
```

Endpoint:

```
/api/notifications/catra_event
```

O monitor recebe eventos como:

```
{
  "device_id": "12345",
  "event": {
    "name": "TURN_RIGHT"
  }
}
```

ou:

```
{
  "device_id": "12345",
  "event": {
    "name": "TURN_LEFT"
  }
}
```

O `device_id` permite descobrir qual driver/catraca deve receber o evento.

interpreta_codigo.py

Responsável por transformar o conteúdo criptografado do QR em dados utilizáveis.

Algoritmo atual:

```
AES-256-ECB
```

Chave:

```
A2_AES_KEY=...
```

A chave deve possuir exatamente 32 bytes ASCII.

O interpretador aceita duas representações do ciphertext:

1. 32 caracteres hexadecimais:

```
b486c08c912653049354b954a1fa2439
```

2. os 16 bytes AES diretamente, inclusive quando preservados como string Latin-1.

Layout descriptografado:

```
bytes 0..3   cpf_base
bytes 4..7   start_timestamp
bytes 8..11  end_timestamp
byte  12     day_mask
bytes 13..15 reservados
```

Os dois dígitos verificadores do CPF são calculados pelo A2.

regras_entrada.py

Aplica as regras de validade do QR.

Atualmente verifica:

- payload AES válida;
- CPF válido no formato esperado;
- `start_timestamp`;
- `end_timestamp`;
- horário atual dentro do intervalo permitido.

O `day_mask` é decodificado, mas ainda não participa da decisão porque o mapeamento oficial dos bits para os dias da semana ainda não foi definido.

s4_client.py

Integra o A2 ao S4 para evidência de mídia.

Em modo real:

```
A2 -> ods.s4.capture.command -> S4
S4 -> reply                  -> A2
S4 -> ods.s4.clip            -> A2
```

Em modo de teste:

```
TESTE_S4=true
```

o A2 gera uma evidência fictícia para permitir testar o restante do sistema sem S4.

database.py

Cria e atualiza automaticamente o schema do SQLite.

Também permite que bancos criados por versões anteriores sejam migrados adicionando as novas colunas sem precisar apagar `acessos.db`.

acessos_api.py

Expõe os logs pela rede local.

Porta padrão:

```
8001
```

Principais endpoints:

```
GET  /health
GET  /acessos
GET  /acessos/{id}
POST /manutencao/limpar
```

Documentação interativa FastAPI:

```
http://IP_DO_A2:8001/docs
```

entrypoint.sh

Inicia e supervisiona separadamente:

```
acessos_api.py
liberar.py
```

Se `liberar.py` sofrer um crash, a API permanece ativa.

Se a API sofrer um crash, o controle de acesso permanece ativo.

O processo que caiu é reiniciado automaticamente.

---

3. Os dois arquivos JSON de configuração

A configuração foi separada propositalmente em duas partes:

```
catracas.json
      ↓
configuração lógica

configCatraca.json
      ↓
configuração física
```

Isso evita misturar a regra:

> *"qual câmera corresponde a qual catraca?"*

com:

> *"qual IP, fabricante e protocolo essa catraca usa?"*

3.1. catracas.json — vínculo câmera → catraca

Esse arquivo define qual catraca pertence a cada `source_id` recebido do I6.

Exemplo:

```
[
  {
    "turnstile_id": "catraca-01",
    "source_id": "camera-01",
    "passage_timeout": 15,
    "max_pending_age": 2
  },
  {
    "turnstile_id": "catraca-02",
    "source_id": "camera-02",
    "passage_timeout": 5,
    "max_pending_age": 2
```

```
  }
]
```

source_id

Identificador da câmera.

Precisa ser exatamente o mesmo `source_id` publicado pelo I6.

Exemplo:

```
I6 publica:
source_id = camera-01

catracas.json:
camera-01 -> catraca-01
```

O A2 não procura a câmera pelo IP.

A associação é feita pelo identificador lógico `source_id`.

turnstile_id

Nome lógico da catraca.

Esse mesmo valor precisa existir em `configCatraca.json`.

passage_timeout

Tempo máximo, em segundos, que o A2 espera uma passagem depois de liberar a catraca.

Exemplo:

```
"passage_timeout": 15
```

Se nenhuma passagem for confirmada nesse intervalo:

```
status = timeout
```

e o acesso não é registrado como passagem concluída.

max_pending_age

Tempo máximo que uma leitura QR pendente ainda é considerada útil.

Exemplo:

```
"max_pending_age": 2
```

Se a catraca estiver ocupada processando uma leitura, o A2 mantém somente a leitura pendente mais recente.

Isso evita formar uma fila longa de QRs antigos.

Fluxo:

```
QR A
QR B
QR C

catraca ocupada

pendente final = QR C
```

Quando o A2 puder processar a leitura, também verifica a idade dela. Se estiver velha demais, descarta.

4. configCatraca.json - configuração física

Esse arquivo descreve como o A2 conversa com os dispositivos.

Estrutura geral:

```
{
  "monitor": {
    "...": "..."
  },
  "devices": [
    {
      "...": "..."
    }
  ]
}
```

4.1. Monitor

Exemplo:

```
{
  "monitor": {
    "bind_host": "0.0.0.0",
    "hostname": "192.168.10.10",
    "port": 8000,
    "path": "api/notifications"
  }
}
```

bind_host

Endereço no qual o servidor HTTP do A2 escuta localmente.

Normalmente:

```
0.0.0.0
```

Isso significa:

> *aceitar conexões recebidas por qualquer interface de rede da máquina.*

hostname

É o endereço que a catraca usará para voltar até o A2.

Portanto deve ser o IP real da máquina/Jetson na LAN.

Exemplo:

```
192.168.10.10
```

Não usar:

```
127.0.0.1
```

porque para a catraca `127.0.0.1` significaria a própria catraca.

Também não usar o IP interno `172.x.x.x` criado pelo Docker.

A catraca deve acessar o IP do host.

port

Porta de callback:

```
8000
```

path

Base do endpoint:

```
api/notifications
```

O monitor do A2 recebe a notificação final em:

```
/api/notifications/catra_event
```

---

5. Configuração real - Control iD

Exemplo de `configCatraca.json` para uma Control iD:

```
{
  "monitor": {
    "bind_host": "0.0.0.0",
    "hostname": "192.168.10.10",
    "port": 8000,
    "path": "api/notifications"
  },

  "devices": [
    {
      "turnstile_id": "catraca-01",
      "model": "controlid_idblock",
      "protocol": "http",
      "host": "192.168.10.101",
      "port": 80,

      "username": "admin",
      "password": "admin",

      "direction": "both",

      "verify_tls": false,
      "request_timeout": 5
    }
  ]
}
```

Campos principais

`turnstile_id`

```
catraca-01
```

deve corresponder ao `turnstile_id` de `catracas.json`.

`model`

```
controlid_idblock
```

seleciona o driver `ControlIDBlock`.

`protocol`

```
http
```

ou:

```
https
```

`host`

IP da catraca:

```
192.168.10.101
```

`port`

Normalmente:

```
80
```

para HTTP, dependendo da configuração do equipamento.

`username` e `password`

Credenciais utilizadas pelo driver para autenticar na catraca.

`direction`

Define qual rotação será liberada:

```
clockwise
anticlockwise
both
```

Normalmente:

```
both
```

permite passagem nos dois sentidos.

`request_timeout`

Timeout das chamadas HTTP feitas pelo driver.

---

6. Como o driver Control iD funciona

Ao conectar, `ControlIDBlock.initialize()` executa aproximadamente:

```
1. login
2. lê informações do equipamento
3. obtém device_id
4. consulta gateway da catraca
5. configura timeout físico
6. configura monitor/callback
```

Chamadas implementadas atualmente:

```
/login.fcgi
/system_information.fcgi
/get_configuration.fcgi
/set_configuration.fcgi
/execute_actions.fcgi
```

Depois da inicialização o driver conhece:

```
device_id
gateway
session_token
```

O `device_id` é registrado no `TurnstileRegistry`.

---

7. Entrada e saída: gateway

Para saber se a pessoa entrou ou saiu, o A2 precisa relacionar a rotação física da catraca com o lado configurado como entrada.

Eventos:

```
TURN_RIGHT -> clockwise
TURN_LEFT  -> anticlockwise
```

A Control iD informa o `gateway` configurado.

Exemplo:

```
gateway = clockwise
```

Então:

```
TURN_RIGHT
    ↓
clockwise == gateway
    ↓
ENTRADA
```

e:

```
TURN_LEFT
    ↓
anticlockwise != gateway
    ↓
SAÍDA
```

Se o `gateway` estiver invertido na instalação física, entrada e saída também aparecerão invertidas no A2.

---

8. Liberação da catraca

Quando o QR é válido:

```
A2
 ↓
driver.release()
```

No driver Control iD é enviada uma ação para:

```
/execute_actions.fcgi
```

com:

```
action = catra
allow  = clockwise / anticlockwise / both
```

Depois disso o A2 entra em:

```
wait_for_passage()
```

e aguarda:

```
TURN_LEFT
TURN_RIGHT
GIVE_UP
timeout
```

---

9. Monitor de passagem

O monitor HTTP fica executando no A2.

Fluxo:

```
Control iD
    │
    │ callback HTTP
    ▼
IP_DO_A2:8000
    │
    ▼
catraca_monitor.py
    │
    ├── lê device_id
    ├── localiza driver
    └── driver.notify_event(...)
```

Exemplo de callback:

```
{
  "device_id": "123456",
  "event": {
    "name": "TURN_RIGHT"
  }
}
```

O registry funciona conceitualmente como:

```
device_id 123456
       ↓
catraca-01
       ↓
ControlIDBlock
```

Por isso vários dispositivos podem usar o mesmo monitor HTTP.

---

10. Reconexão automática

Uma catraca offline não derruba o A2.

Cada catraca possui um worker de conexão independente.

Exemplo:

```
A2 inicia
 │
 ├── API de logs ONLINE
 ├── Kafka ONLINE
 └── catraca-01 OFFLINE
          │
          ├── tenta conectar
          ├── falha
          ├── espera 5 s
          └── tenta novamente
```

Configuração:

```
A2_TURNSTILE_RETRY_SECONDS=5
```

Quando a catraca voltar:

```
[catraca-01] Tentando reconectar...
[catraca-01] ONLINE.
```

Não é necessário reiniciar o Docker.

Se uma comunicação falhar durante `release()`, a catraca volta para `OFFLINE` e o worker começa novamente o processo de conexão.

---

11. Reconexão do monitor

O monitor da porta 8000 também possui retry.

Configuração:

```
A2_MONITOR_RETRY_SECONDS=5
```

Se a porta não puder ser aberta:

```
[Monitor] OFFLINE
[Monitor] Nova tentativa em 5s
```

A API de logs continua funcionando.

12. Como adicionar suporte a outra marca de catraca

A lógica de negócio do A2 não conhece detalhes da Control iD.

Ela trabalha contra uma interface prática de driver.

Um novo driver deve implementar, em essência:

```
class MinhaCatraca:
    def __init__(self, config, passage_timeout):
        ...

    def initialize(self, monitor_config):
        ...

    def release(self):
        ...

    def notify_event(self, event_name):
        ...

    def wait_for_passage(self):
        ...
```

Se o equipamento não trabalhar com callbacks HTTP, `wait_for_passage()` pode consultar/pollar a API própria do fabricante.

O retorno esperado de `wait_for_passage()` é:

```
{
    "status": "passed",
    "direction": "entrada"
}
```

ou:

```
{
    "status": "passed",
    "direction": "saida"
}
```

ou:

```
{
    "status": "timeout",
    "direction": None
}
```

ou:

```
{
    "status": "give_up",
    "direction": None
}
```

Depois é necessário registrar o novo modelo em:

```
create_turnstile_driver()
```

Exemplo conceitual:

```
if (
    model == "fabricante_x"
    and protocol == "http"
):
    return FabricanteX(
        hardware_config,
        passage_timeout
    )
```

Assim o restante do A2 permanece inalterado:

```
Kafka
QR
regras
S4
logs
API
```

não precisam saber qual fabricante está sendo utilizado.

---

13. Simulação sem hardware físico

Existem três formas de teste.

13.1. Simulação automática simples

Em `configCatraca.json`:

```
{
  "devices": [
    {
      "turnstile_id": "catraca-01",
      "model": "simulated",
      "protocol": "simulation",
      "simulation": {
        "pass_after": 2,
        "direction": "entrada"
      }
    }
  ]
}
```

Nesse modo a passagem é simulada automaticamente após `pass_after`.

13.2. Simulador visual local

Quando:

```
A2_SIMULATION=true
```

e o dispositivo é `simulated`, o A2 pode usar o simulador visual local.

Configurações:

```
A2_SIMULATOR_HOST=127.0.0.1
A2_SIMULATOR_PORT=8765
```

13.3. Simulador externo — celular ou computador

O driver `mobile` não depende de Android.

`catraca_celular.py` pode rodar em outro computador.

Exemplo:

```
PC A2/Jetson
192.168.10.10

PC simulando catraca
192.168.10.150
```

Configuração:

```
{
  "monitor": {
    "bind_host": "0.0.0.0",
    "hostname": "192.168.10.10",
    "port": 8000,
    "path": "api/notifications"
  },

  "devices": [
    {
      "turnstile_id": "catraca-01",
      "model": "mobile",
      "protocol": "http",
      "host": "192.168.10.150",
      "port": 8080,
      "device_id": "mobile-catraca-01",
      "gateway": "clockwise",
      "direction": "both",
      "request_timeout": 5
    }
  ]
}
```

Fluxo:

```
A2
 │
 │ POST /configure
 ▼
PC catraca
 │
 │ POST /release
 ▼
interface aguarda ação

usuário escolhe Entrada/Saída
 │
 ▼
callback -> A2:8000
```

---

14. Evento recebido do I6

O A2 consome:

```
ods.i6.code_read
```

Campos usados:

```
message_id
captured_at
source_id
payload.frame_index
payload.decoded_data
```

Exemplo:

```
{
  "message_type": "event",
  "message_id": "550e8400-e29b-41d4-a716-446655440000",
  "schema": "ods.i6.code_read",
  "schema_version": "1.0",
  "producer": "I6",
  "captured_at": "2026-10-30T13:00:00.000Z",
  "source_id": "camera-01",

  "payload": {
    "frame_index": 9031,
    "code_type": "QR_CODE",
    "decoded_data": "b486c08c912653049354b954a1fa2439",
    "bbox": [812, 300, 1010, 498]
  }
}
```

`frame_index` nasce no P4 e é propagado pelo I6.

O A2 não consulta P4 diretamente.

Fluxo correto:

```
P4
 │ frame_index = 9031
 │ captured_at = X
 ▼
I6
 │ lê QR no frame 9031
 ▼
A2
```

15. Deduplicação Kafka

O A2 utiliza `message_id` como chave principal de deduplicação.

Configuração:

```
KAFKA_DEDUP_TTL=60
```

Se a mesma mensagem for entregue novamente dentro desse período, ela é ignorada.

Caso `message_id` não exista, existe uma chave de fallback baseada em:

```
source_id
decoded_data
captured_at
```

16. Kafka

Configurações:

```
KAFKA_BOOTSTRAP_SERVERS=IP_OU_HOST_DO_BROKER:PORTA
KAFKA_TOPIC=ods.i6.code_read
KAFKA_GROUP_ID=a2-controle-acesso
KAFKA_AUTO_OFFSET_RESET=latest
KAFKA_DEDUP_TTL=60
```

`KAFKA_BOOTSTRAP_SERVERS` deve ser substituído pelo broker real da integração.

Com:

```
KAFKA_AUTO_OFFSET_RESET=latest
```

um grupo sem offset salvo começa a consumir mensagens novas, não o histórico anterior.

Durante teste, portanto:

```
1. subir A2
2. confirmar consumidor
3. depois publicar/ler QR
```

17. Validação do QR

Fluxo:

```
decoded_data
    ↓
interpreta_codigo.py
    ↓
```

```
AES-256-ECB
    ↓
CPF + início + fim + day_mask
    ↓
regras_entrada.py
```

O acesso só continua se:

```
start_timestamp <= agora <= end_timestamp
```

A chave atual de teste normalmente é:

```
A2_AES_KEY=aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
```

Ela deve ser substituída pela chave real antes do deploy.

---

18. Evidência S4

Quando o QR é válido, antes da liberação física o A2 solicita um snapshot referente ao instante original da leitura:

```
captured_at do I6
```

Comando:

```
ods.s4.capture.command
```

Conceitualmente:

```
{
  "schema": "ods.s4.capture.command",
  "message_type": "command",
  "producer": "A2",
  "target": "S4",

  "payload": {
    "source_id": "camera-01",
    "media_type": "SNAPSHOT",
    "at": "captured_at do I6",
    "event_id": null,
    "label": "acesso CPF ..."
  }
}
```

O A2 não salva JPEG ou MP4.

Ele salva apenas:

```
clip_id
media_uri
hash_sha256
expires_at
status
```

---

19. S4 em modo de teste

Enquanto S4 não estiver disponível:

```
TESTE_S4=true
```

O A2 cria:

```
media_status = READY
clip_id      = UUID fictício
media_uri    = teste-s4://...
hash_sha256  = hash fictício
expires_at   = data fictícia
```

Quando o S4 real estiver disponível:

```
TESTE_S4=false
```

Tópicos configuráveis:

```
S4_CAPTURE_COMMAND_TOPIC=ods.s4.capture.command
S4_CAPTURE_REPLY_TOPIC=ods.s4.capture.command.reply
S4_CLIP_TOPIC=ods.s4.clip
```

---

20. Registro no SQLite

O banco fica, no host:

```
./data/acessos.db
```

e dentro do container:

```
/app/data/acessos.db
```

Campos principais:

```
id
cpf
direction
occurred_at
source_id
turnstile_id

i6_message_id
frame_index
captured_at

media_request_id
media_clip_id
media_type
media_uri
media_hash_sha256
media_expires_at
media_status
```

O acesso atualmente é registrado quando uma passagem física é confirmada.

Timeout, desistência ou QR inválido não são registrados como uma passagem concluída na tabela `access_logs`.

---

21. API local de logs

A API roda por padrão em:

```
8001
```

Teste:

```
http://localhost:8001/health
```

De outro computador na LAN:

```
http://IP_DO_A2:8001/health
```

Listar acessos:

```
http://IP_DO_A2:8001/acessos
```

Filtros:

```
/acessos?cpf=12345678909
/acessos?direction=entrada
/acessos?direction=saida
/acessos?source_id=camera-01
/acessos?turnstile_id=catraca-01
/acessos?media_status=READY
```

Registro específico:

```
/acessos/15
```

Swagger/FastAPI:

```
http://IP_DO_A2:8001/docs
```

22. Retenção dos logs

Configuração padrão:

```
A2_LOG_RETENTION_DAYS=30
```

Intervalo de limpeza:

```
A2_LOG_CLEANUP_INTERVAL_SECONDS=21600
```

A limpeza é automática.

Também existe:

```
POST /manutencao/limpar
```

23. Rede recomendada

Durante desenvolvimento é possível usar os IPv4 fornecidos pelo DHCP.

Para instalação definitiva é recomendado reservar/fixar os endereços.

Exemplo de organização:

```
Rede: 192.168.10.0/24

Roteador/AP           192.168.10.1
Jetson/A2             192.168.10.10

catraca-01            192.168.10.101
catraca-02            192.168.10.102
catraca-03            192.168.10.103

PC simulador          192.168.10.150
```

Máscara:

```
255.255.255.0
```

O importante é que os dispositivos consigam se alcançar nas duas direções.

Portas do A2:

```
8000 -> callbacks das catracas
8001 -> API local de logs
```

Porta típica do simulador externo:

```
8080
```

Para Control iD, a porta depende da configuração HTTP/HTTPS do equipamento.

24. Docker

O A2 utiliza um único container para os dois processos principais:

```
container ods-a2
 ├── liberar.py
 └── acessos_api.py
```

Kafka permanece externo.

I6 permanece externo.

S4 permanece externo.

Catracas permanecem dispositivos externos.

O Docker publica:

```
8000:8000
8001:8001
```

Volumes:

```
./data:/app/data
./catracas.json:/app/catracas.json:ro
./configCatraca.json:/app/configCatraca.json:ro
```

O SQLite fica persistente mesmo após rebuild do container.

---

25. Como colocar para rodar

Na pasta do A2:

```
cp .env.example .env
mkdir -p data
```

Edite:

```
.env
catracas.json
configCatraca.json
```

Depois:

```
docker compose up -d --build
```

Ver containers:

```
docker ps
```

Ver logs:

```
docker logs -f ods-a2
```

Parar:

```
docker compose down
```

Reiniciar:

```
docker compose restart
```

Rebuild:

```
docker compose up -d --build
```

---

26. Supervisão dos processos

O `entrypoint.sh` mantém API e A2 independentes.

Situação:

```
liberar.py caiu
```

Resultado:

```
API :8001 continua funcionando
entrypoint reinicia liberar.py
```

Situação:

```
acessos_api.py caiu
```

Resultado:

```
liberar.py continua funcionando
entrypoint reinicia API
```

Intervalo:

```
A2_PROCESS_RESTART_SECONDS=2
```

---

27. Sequência recomendada de teste de integração

Primeiro valide a rede:

```
A2 -> catraca
catraca -> A2
```

Depois valide:

```
http://IP_DO_A2:8001/health
```

Então:

```
1. subir simulador/catraca
2. subir A2
3. abrir docker logs -f ods-a2
4. subir I6
5. mostrar QR válido
6. confirmar "Código válido"
7. confirmar liberação da catraca
8. realizar Entrada ou Saída
9. confirmar callback
10. abrir /acessos
11. conferir log e evidência
```

Fluxo esperado:

```
I6
 ↓
Kafka
 ↓
A2
 ↓
QR válido
 ↓
S4 teste/real
 ↓
catraca
 ↓
passagem
 ↓
callback
 ↓
SQLite
```

```
 ↓
API
```

---

28. Diagnóstico rápido

A2 não recebe QR

Verificar:

```
KAFKA_BOOTSTRAP_SERVERS
KAFKA_TOPIC
KAFKA_GROUP_ID
```

e os logs:

```
docker logs -f ods-a2
```

QR chega, mas nenhuma catraca é encontrada

Provável diferença entre:

```
message.source_id
```

e:

```
catracas.json -> source_id
```

Catraca fica OFFLINE

Verificar:

```
IP
porta
firewall
credenciais
modelo
protocolo
```

O A2 continuará tentando reconectar.

A2 libera, mas não recebe passagem

Verificar:

```
monitor.hostname
porta 8000
firewall
callback configurado na catraca
device_id
```

API não abre

Testar dentro da máquina:

```
http://localhost:8001/health
```

e verificar:

```
docker ps
docker logs ods-a2
```

Entrada e saída aparecem invertidas

Verificar o `gateway` físico/configurado da catraca.

---

29. O que está pronto atualmente

Já implementado:

```
Kafka / ods.i6.code_read
múltiplas câmeras
múltiplas catracas
source_id -> turnstile_id
deduplicação
latest pending QR por catraca
timeout de leitura pendente
AES-256-ECB
CPF
janela temporal
driver Control iD
drivers de simulação
monitor HTTP
entrada/saída por rotação
timeout de passagem
reconexão automática
SQLite
migração automática de schema
retenção de logs
API local
frame_index/message_id/captured_at no log
S4 em modo teste
estrutura para S4 real
Docker
supervisão independente de processos
```

---

30. Pendências / pontos ainda não definitivos

Regra anti-reentrada

Ainda não foi implementada a regra:

```
entrou
 ↓
não pode entrar novamente
 ↓
até registrar uma saída
```

Essa regra deve ser adicionada usando estado persistente no SQLite para sobreviver a reinicializações.

day_mask

É lido do QR, mas ainda não participa da autorização porque falta definir oficialmente a correspondência dos bits com os dias da semana.

S4 real

A estrutura já existe, mas enquanto:

```
TESTE_S4=true
```

a evidência é simulada.

Quando o S4 real entrar na integração, é necessário confirmar os tópicos Kafka finais e a forma de resolução de `media_uri`.

Chave AES

A chave:

```
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
```

é apenas de teste.

Hardware real

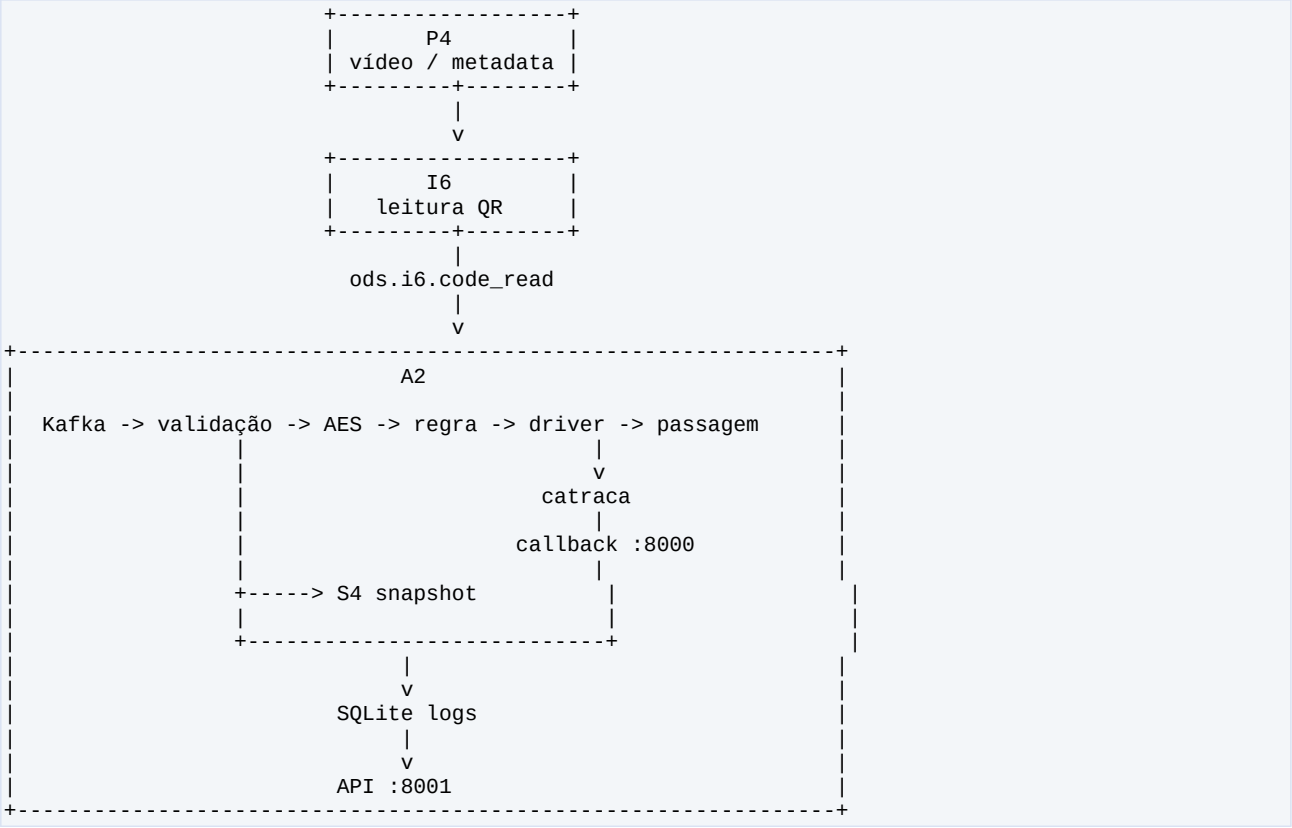
Atualmente o driver real implementado é:

```
Control iD / iDBlock
```

Outros fabricantes precisam de um novo driver, mas a arquitetura já está preparada para isso.

---

31. Resumo da arquitetura



A regra central de integração é:

```
source_id identifica o ponto de acesso
turnstile_id identifica a catraca
device_id identifica o equipamento físico no callback
message_id identifica a mensagem do I6
frame_index identifica o quadro original
captured_at identifica o instante original da captura
clip_id identifica a evidência do S4
```

Com esses identificadores é possível rastrear um acesso desde o quadro da câmera até o registro final no banco.